`views::concat`

Document #:	P2542R0
Date:	2021-02-14
Project:	Programming Language C++
Audience:	SG9, LEWG

Reply-to:	Hui Xie <hui.xie1990@gmail.com> S. Levent Yilmaz <levent.yilmaz@gmail.com>

Contents

§ 1 Revision History

This is the initial revision.

§ 2 Abstract

This paper proposes `views::concat` as very briefly introduced in Section 4.7 of [P2214R1]. It is a view factory that takes an arbitrary number of ranges as an argument list, and provides a view that starts at the first element of the first range, ends at the last element of the last range, with all range elements sequenced in between respectively in the order given in the arguments, effectively concatenating, or chaining together the argument ranges.

§ 3 Motivation and Examples

Before

```
std::vector<int> v1{1,2,3}, v2{4,5}, v3{};
std::array  a{6,7,8};
int s = 9;
std::cout << std::format("[{:n}, {:n}, {:n},
        {:n}, {}]\n",
                v1, v2, v3, a, s);
// output:  [1, 2, 3, 4, 5, , 6, 7, 8, 9]
```

After

```
std::vector<int> v1{1,2,3}, v2{4,5}, v3{};
std::array  a{6,7,8};
auto s = std::views::single(9);
std::cout << std::format("{}\n",
                std::views::concat(v1, v2, v3, a,
        s));
// output:  [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Before

```
class Foo;

class Bar{
  const Foo& getFoo() const;
};

class MyClass{
  Foo foo_;
  std::vector<Bar> bars_;
public:
  auto getFoos () const{
    std::vector<std::reference_wrapper<Foo
        const>> fooRefs;
    fooRefs.reserve(bars_.size() + 1);
    fooRefs.push_back(std::cref(foo_));
    std::ranges::transform(bars_,
        std::back_inserter(bars),
          [](const Bar& bar){
            return std::cref(bar.getFoo());
          });
    return fooRefs;
  }
};

// user
for(const auto& fooRef: myClass.getFoos() |
        views::filter(pred)){
  // `foo` is std::reference_wrapper<Foo const>
  // ...
}
```

After

```
class Foo;

class Bar{
  const Foo& getFoo() const;
};

class MyClass{
  Foo foo_;
  std::vector<Bar> bars_;

  auto getFoos () const{
    using views = std::views;
    return
        views::concat(views::single(std::cref(foo_),
        bars_ | views::transform(&Bar::getFoo)));
  }
};

// user
for(const auto& foo: myClass.getFoos() |
        views::filter(pred)){
  // use foo
}
```

The first example shows that dealing with multiple ranges require care even in the simplest of cases: The “Before” version manually concatenates all the ranges in a formatting string, but empty ranges aren’t handled and the result contains an extra comma and a space. With `concat_view`, empty ranges are handled per design, and the construct expresses the intent cleanly and directly.

In the second example, the user has a class composed of fragmented and indirect data. They want to implement a member function that provides a range-like view to all this data sequenced together in some order, and without creating any copies. The “Before” implementation is needlessly complex and creates a temporary container. `concat_view` based implementation is a neat one-liner.

§ 4 Design

This is a generator factory as described in [P2214R1] Section 4.7. As such, it can not be piped to. It takes the list of ranges to concatenate as arguments to `ranges::concat_view` constructor, or to `ranges::views::concat` customization point object.

§ 4.1 `concatable`-ity of ranges

Adaptability of any given two or more distinct `ranges` into a sequence that itself models a `range`, depends on the compatibility of the reference and the value types of these ranges. A precise formulation is made in terms of `std::common_reference_t` and `std::common_type_t`, and is captured by the exposition only concept `concatable`. See Wording. Proposed `concat_view` is then additionally constrained by this concept. (Note that, this is an improvement over [range-v3] `concat_view` which lacks such constraints, and fails with hard errors instead.)

§ 4.1.1 `reference`

The `reference` type is the `common_reference_t` of all underlying range's `range_reference_t`. In addition, as the result of `common_reference_t` is not necessarily a reference type, an extra constraint is needed to make sure that each underlying range's `range_reference_t` is convertible to that common reference.

§ 4.1.2 `value_type`

To support the cases where underlying ranges have proxy iterators, such as `zip_view`, the `value_type` cannot simply be the `remove_cvref_t` of the `reference` type, and it needs to respect underlying ranges' `value_type`. Therefore, in this proposal the `value_type` is defined as the `common_type_t` of all underlying range's `range_value_t`.

§ 4.1.3 `range_rvalue_reference_t`

To make `concat_view`'s iterator's `iter_move` behave correctly for the cases where underlying iterators customise `iter_move`, such as `zip_view`, `concat_view` has to respect those customizations. Therefore, `concat_view` requires `common_reference_t` of all underlying ranges's `range_rvalue_reference_t` exist and can be converted to from each underlying range's `range_rvalue_reference_t`.

§ 4.1.4 Unsupported Cases and Potential Extensions for the Future

Common type and reference based `concatable` logic is a practical and convenient solution that satisfies the motivation as outlined, and is what the authors propose in this paper. However, there are several potentially important use cases that get left out:

1. Concatenating ranges of different subclasses, into a range of their common base.
2. Concatenating ranges of unrelated types into a range of a user-determined common type, e.g. a `std::variant`.

Here is an example of the first case where `common_reference` formulation can manifest a rather counter-intuitive behavior: Let `D1` and `D2` be two types only related by their common base `B`, and `d1`, `d2`, and `b` be some range of these types, respectively. `concat(b, d1, d2)` is a well-formed range of `B` by the current formulation, suggesting such usage is supported. However, a mere reordering of the sequence, say to `concat(d1, d2, b)`, yields one that is not.

The authors believe that such cases should be supported, but can only be done so via an adaptor that needs at least one explicit type argument at its interface. A future extension may satisfy these use cases, for example a `concat_as` view, or by even generally via an `as` view that is a type-generalized version of the `as_const` view of [P2278R1].

§ 4.2 Zero or one view

— No argument `views::concat()` is ill-formed. It can not be a `views::empty<T>` because there is no reasonable way to determine an element type `T`.

— Single argument `views::concat(r)` is expression equivalent to `views::all(r)`, which intuitively follows.

§ 4.3 Hidden *O(N)* time complexity for *N* adapted ranges

Time complexities as required by the `ranges` concepts are formally expressed with respect to the total number of elements (the size) of a given range, and not to the statically known parameters of that range. Hence, the complexity of `concat_view` or its iterators' operations are documented to be constant time, even though some of these are a linear function of the number of ranges it concatenates which is a statically known parameter of this view.

Some examples of these operations for `concat_view` are copy, `begin` and `size`, and its iterators' increment, decrement, advance, distance. This characteristic (but not necessarily the specifics) are very much similar to the other n-ary adaptors like `zip_view` [P2321R2] and `cartesian_view` [P2374R3].

§ 4.4 Borrowed vs Cheap Iterator

`concat_view` can be designed to be a `borrowed_range`, if all the underlying ranges are. However, this requires the iterator implementation to contain a copy of all iterators and sentinels of all underlying ranges at all times (just like that of `views::zip` [P2321R2]). On the other hand (unlike `views::zip`), a much cheaper implementation can satisfy all the proposed functionality provided it is permitted to be unconditionally not borrowed. This implementation would maintain only a single active iterator at a time and simply refers to the parent view for the bounds.

Experience shows the borrowed-ness of `concat` is not a major requirement; and the existing implementation in [range-v3] seems to have picked the cheaper alternative. This paper proposes the same.

§ 4.5 Common Range

`concat_view` can be `common_range` if the last underlying range models either

- `common_range`, or
- `random_access_range && sized_range`

§ 4.6 Bidirectional Range

`concat_view` can model `bidirectional_range` if the underlying ranges satisfy the following conditions:

- Every underlying range models `bidirectional_range`
- If the iterator is at nth range's begin position, after `operator--` it should go to (n-1)th's range's end-1 position. This means, the (n-1)th range has to support either
  - `common_range && bidirectional_range`, so the position can be reached by `--ranges::end(n-1th range)`, assuming n-1th range is not empty, or
  - `random_access_range && sized_range`, so the position can be reached by `ranges::begin(n-1th range) + (ranges::size(n-1th range) - 1)` in constant time, assuming n-1th range is not empty.

  Note that the last underlying range does not have to satisfy this constraint because n-1 can never be the last underlying range. If neither of the two constraints is satisfied, in theory we can cache the end-1 position for every single underlying range inside the `concat_view` itself. But the authors do not consider this type of ranges as worth supporting bidirectional

In the `cancat` implementation in [range-v3], `operator--` is only constrained on all underlying ranges being `bidirectional_range` on the declaration, but its implementation is using `ranges::next(ranges::begin(r), ranges::end(r))` which implicitly requires random access to make the operation constant time. So it went with the second constraint. In this paper, both are supported.

§ 4.7 Random Access Range

`concat_view` can be `random_access_range` if all the underlying ranges model `random_access_range` and `sized_range`.

§ 4.8 Sized Range

`concat_view` can be `sized_range` if all the underlying ranges model `sized_range`

4.9 Implementation experience

`views::concat` has been implemented in [range-v3], with equivalent semantics as proposed here. The authors also implemented a version that directly follows the proposed wording below without any issue [ours].

5 Wording

5.1 Addition to `<ranges>`

Add the following to 24.2 [ranges.syn], header `<ranges>` synopsis:

```
// [...]
namespace std::ranges {
  // [...]

  // [range.concat], concat view
  template <input_range... Views>
    requires see below
  class concat_view;

  namespace views {
    inline constexpr unspecified concat = unspecified;
  }

}
```

5.2 Range adaptor helpers [range.adaptor.helpers]

This paper applies the same following changes as in [P2374R3]. If [P2374R3] is merged into the standard, the changes in this section can be dropped.

New section after Non-propagating cache 24.7.4 [range.nonprop.cache]. Move the definitions of *`tuple-or-pair`*, *`tuple-transform`*, and *`tuple-for-each`* from Class template `zip_view` 24.7.19.2 [range.zip.view] to this section:

```
namespace std::ranges {
    template <class... Ts>
    using tuple-or-pair = see-below;                    // exposition only

    template<class F, class Tuple>
    constexpr auto tuple-transform(F&& f, Tuple&& tuple) { // exposition only
        return apply([&]<class... Ts>(Ts&&... elements) {
            return tuple-or-pair<invoke_result_t<F&, Ts>...>(
                invoke(f, std::forward<Ts>(elements))...
            );
        }, std::forward<Tuple>(tuple));
    }

    template<class F, class Tuple>
    constexpr void tuple-for-each(F&& f, Tuple&& tuple) { // exposition only
        apply([&]<class... Ts>(Ts&&... elements) {
            (invoke(f, std::forward<Ts>(elements)), ...);
        }, std::forward<Tuple>(tuple));
    }
}
```

Given some pack of types `Ts`, the alias template *`tuple-or-pair`* is defined as follows:

1. If `sizeof...(Ts)` is `2`, *`tuple-or-pair`*`<Ts...>` denotes `pair<Ts...>`.
2. Otherwise, *`tuple-or-pair`*`<Ts...>` denotes `tuple<Ts...>`.

5.3 `concat`

Add the following subclause to 24.7 [range.adaptors].

24.7.? Concat view [range.concat]

24.7.?.1 Overview [range.concat.overview]

1 `concat_view` presents a `view` that concatenates all the underlying ranges.

2 The name `views::concat` denotes a customization point object (16.3.3.3.6 [customization.point.object]). Given a pack of subexpressions `Es...`, the expression `views::concat(Es...)` is expression-equivalent to

(2.1) — `views::all(Es...)` if `Es` is a pack with only one element and `views::all(Es...)` is a well formed expression,

(2.2) — otherwise, `concat_view(Es...)` if this expression is well formed,

(2.3) — otherwise, ill-formed.

[*Example:*

```
std::vector<int> v1{1,2,3}, v2{4,5}, v3{};
std::array  a{6,7,8};
auto s = std::views::single(9);
for(auto&& i : std::views::concat(v1, v2, v3, a, s)){
  std::cout << i << ' '; // prints: 1 2 3 4 5 6 7 8 9
}
```

— *end example*]

24.7.?.2 Class template `concat_view` [range.concat.view]

```
namespace std::ranges {

  template <class... Rs>
  concept concatable = see below;             // exposition only

  template <class... Rs>
  concept concat-random-access =              // exposition only
    ((random_access_range<Rs> && sized_range<Rs>)&&...);

  template <class R>
  concept constant-time-reversible =          // exposition only
    (bidirectional_range<R> && common_range<R>) ||
    (sized_range<R> && random_access_range<R>);

  template <class... Rs>
  concept concat-bidirectional = see below;   // exposition only

  template <input_range... Views>
    requires (view<Views> && ...) && (sizeof...(Views) > 0) &&
              concatable<Views...>
  class concat_view : public view_interface<concat_view<Views...>> {
    tuple<Views...> views_ = tuple<Views...>(); // exposition only

    template <bool Const>
    class iterator;                           // exposition only

  public:
    constexpr concat_view() requires(default_initializable<Views>&&...) = default;

    constexpr explicit concat_view(Views... views);

    constexpr iterator<false> begin() requires(!(simple-view<Views> && ...));

    constexpr iterator<true> begin() const
      requires((range<const Views> && ...) && concatable<const Views...>);

    constexpr auto end() requires(!(simple-view<Views> && ...));

    constexpr auto end() const requires(range<const Views>&&...);

    constexpr auto size() requires(sized_range<Views>&&...);
```

```
    constexpr auto size() const requires(sized_range<const Views>&&...);
  };

  template <class... R>
    concat_view(R&&...) -> concat_view<views::all_t<R>...>;
}
```

```
template <class... Rs>
concept concatable = see below;
```

1 The exposition-only *concatable* concept is equivalent to:

```
template <class... Rs>
concept concatable = requires {
    typename common_reference_t<range_reference_t<Rs>...>;
    typename common_type_t<range_value_t<Rs>...>;
    typename common_reference_t<range_rvalue_reference_t<Rs>...>;
} &&
(convertible_to<range_reference_t<Rs>,
    common_reference_t<range_reference_t<Rs>...>> && ...) &&
(convertible_to<range_rvalue_reference_t<Rs>,
    common_reference_t<range_rvalue_reference_t<Rs>...>> && ...) &&
common_reference_with<common_reference_t<range_reference_t<Rs>...>&&,
    common_type_t<range_value_t<Rs>...>&> &&
common_reference_with<common_reference_t<range_reference_t<Rs>...>&&,
    common_reference_t<range_rvalue_reference_t<Rs>...>&&> &&
common_reference_with<common_reference_t<range_rvalue_reference_t<Rs>...>&&,
    const common_type_t<range_value_t<Rs>...>&>;
```

```
template <class... Rs>
concept concat-bidirectional = see below;
```

2 The pack `Rs...` models *concat-bidirectional* if,

(2.1) — Last element of `Rs...` models `bidirectional_range`,

(2.2) — And, all except the last element of `Rs...` model *constant-time-reversible*.

```
constexpr explicit concat_view(Views... views);
```

3 *Effects*: Initializes *views_* with `std::move(views)...`.

```
constexpr iterator<false> begin() requires(!(simple-view<Views> && ...));
constexpr iterator<true> begin() const
  requires((range<const Views> && ...) && concatable<const Views...>);
```

4 *Effects*: Let *is-const* be `true` for const-qualified overload, and `false` otherwise. Equivalent to:

```
iterator<is-const> it{this, in_place_index<0>, ranges::begin(get<0>(views_))};
it.template satisfy<0>();
return it;
```

```
constexpr auto end() requires(!(simple-view<Views> && ...));
constexpr auto end() const requires(range<const Views>&&...);
```

5 *Effects*: Let *is-const* be `true` for const-qualified overload, and `false` otherwise, and let *last-view* be the last element of the pack `const Views...` for const-qualified overload, and the last element of the pack `Views...` otherwise. Equivalent to:

```
if constexpr (common_range<last-view>) {
    constexpr auto N = sizeof...(Views);
    return iterator<is-const>{this, in_place_index<N - 1>,
                      ranges::end(get<N - 1>(views_))};
} else if constexpr (random_access_range<last-view> && sized_range<last-view>) {
    constexpr auto N = sizeof...(Views);
    return iterator<is-const>{
        this, in_place_index<N - 1>,
        ranges::begin(get<N - 1>(views_)) + ranges::size(get<N - 1>(views_))};
} else {
```

```
    return default_sentinel;
}
```

```
constexpr auto size() requires(sized_range<Views>&&...);
constexpr auto size() const requires(sized_range<const Views>&&...);
```

6 *Effects*: Equivalent to:

```
return apply(
    [](auto... sizes) {
        using CT = make_unsigned_t<common_type_t<decltype(sizes)...>>;
        return (CT{0} + ... + CT{sizes});
    },
    tuple-transform(ranges::size, views_));
```

§ 24.7.?.3 Class concat_view::iterator [range.concat.iterator]

```
namespace std::ranges{

  template <input_range... Views>
    requires (view<Views> && ...) && (sizeof...(Views) > 0) &&
             concatable<Views...>
  template <bool Const>
  class concat_view<Views...>::iterator {

  public:
    using value_type = common_type_t<range_value_t<maybe-const<Const, Views>>...>;
    using difference_type = common_type_t<range_reference_t<maybe-const<Const, Views>>...>;
    using iterator_concept = see below;
    using iterator_category = see below;                  // not always present.

  private:
    using base_iter =                                     // exposition only
      variant<iterator_t<maybe-const<Const, Views>>...>;

    maybe-const<Const, concat_view>* parent_ = nullptr;   // exposition only
    base_iter it_ = base_iter();                          // exposition only

    friend class iterator<!Const>;
    friend class concat_view;

    template <std::size_t N>
    constexpr void satisfy();                             // exposition only

    template <std::size_t N>
    constexpr void prev();                                // exposition only

    template <std::size_t N>
    constexpr void advance_fwd(difference_type offset, difference_type steps); // exposition only

    template <std::size_t N>
    constexpr void advance_bwd(difference_type offset, difference_type steps); // exposition only

    template <class... Args>
    explicit constexpr iterator(
                maybe-const<Const, concat_view>* parent,
                Args&&... args)
        requires constructible_from<base_iter, Args&&...>;  // exposition only

  public:

    iterator() requires(default_initializable<iterator_t<maybe-const<Const, Views>>>&&...) =
        default;

    constexpr iterator(iterator<!Const> i)
        requires Const &&
        (convertible_to<iterator_t<Views>, iterator_t<maybe-const<Const, Views>>>&&...);

    constexpr decltype(auto) operator*() const;
```

```
  constexpr iterator& operator++();

  constexpr void operator++(int);

  constexpr iterator operator++(int)
      requires(forward_range<maybe-const<Const, Views>>&&...);

  constexpr iterator& operator--()
      requires concat-bidirectional<maybe-const<Const, Views>...>;

  constexpr iterator operator--(int)
      requires concat-bidirectional<maybe-const<Const, Views>...>

  constexpr iterator& operator+=(difference_type n)
      requires concat-random-access<maybe-const<Const, Views>...>;

  constexpr iterator& operator-=(difference_type n)
      requires concat-random-access<maybe-const<Const, Views>...>;

  constexpr decltype(auto) operator[](difference_type n) const
      requires concat-random-access<maybe-const<Const, Views>...>;

  friend constexpr bool operator==(const iterator& x, const iterator& y)
      requires(equality_comparable<iterator_t<maybe-const<Const, Views>>>&&...);

  friend constexpr bool operator==(const iterator& it, default_sentinel_t);

  friend constexpr bool operator<(const iterator& x, const iterator& y)
      requires(random_access_range<maybe-const<Const, Views>>&&...);

  friend constexpr bool operator>(const iterator& x, const iterator& y)
      requires(random_access_range<maybe-const<Const, Views>>&&...);

  friend constexpr bool operator<=(const iterator& x, const iterator& y)
      requires(random_access_range<maybe-const<Const, Views>>&&...);

  friend constexpr bool operator>=(const iterator& x, const iterator& y)
      requires(random_access_range<maybe-const<Const, Views>>&&...);

  friend constexpr auto operator<=>(const iterator& x, const iterator& y)
      requires((random_access_range<maybe-const<Const, Views>> &&
       three_way_comparable<maybe-const<Const, Views>>)&&...);

  friend constexpr iterator operator+(const iterator& it, difference_type n)
      requires concat-random-access<maybe-const<Const, Views>...>;

  friend constexpr iterator operator+(difference_type n, const iterator& it)
      requires concat-random-access<maybe-const<Const, Views>...>;

  friend constexpr iterator operator-(const iterator& it, difference_type n)
      requires concat-random-access<maybe-const<Const, Views>...>;

  friend constexpr difference_type operator-(const iterator& x, const iterator& y)
      requires concat-random-access<maybe-const<Const, Views>...>;

  friend constexpr difference_type operator-(const iterator& x, default_sentinel_t)
      requires concat-random-access<maybe-const<Const, Views>...>;

  friend constexpr difference_type operator-(default_sentinel_t, const iterator& x)
      requires concat-random-access<maybe-const<Const, Views>...>;

  friend constexpr decltype(auto) iter_move(iterator const& it) noexcept(see below);

  friend constexpr void iter_swap(const iterator& x, const iterator& y) noexcept(see below)
      requires see below;
};

}
```

1 *iterator*::iterator_concept is defined as follows:

(1.1) — If *concat-random-access*<*maybe-const*<Const, Views>...> is modeled, then iterator_concept denotes random_access_iterator_tag.

(1.2) — Otherwise, if *concat-bidirectional*<*maybe-const*<Const, Views>...> is modeled, then iterator_concept denotes bidirectional_iterator_tag.

(1.3) — Otherwise, if (forward_range<*maybe-const*<Const, Views>> && ...) is modeled, then iterator_concept denotes forward_iterator_tag.

(1.4) — Otherwise, iterator_concept denotes input_iterator_tag.

2 The member typedef-name iterator_category is defined if and only if (forward_range<*maybe-const*<Const, Views>>&&...) is modeled. In that case, iterator::iterator_category is defined as follows:

(2.1) — If is_lvalue_reference<reference> is false, then iterator_category denotes input_iterator_tag

(2.2) — Otherwise, let Cs denote the pack of types iterator_traits<iterator_t<*maybe-const*<Const, Views>>>::iterator_category....

(2.2.1) — If (derived_from<Cs, random_access_iterator_tag> && ...) && *concat-random-access*<*maybe-const*<Const, Views>...> is true, iterator_category denotes random_access_iterator_tag.

(2.2.2) — If (derived_from<Cs, bidirectional_iterator_tag> && ...) && *concat-bidirectional*<*maybe-const*<Const, Views>...> is true, iterator_category denotes bidirectional_iterator_tag.

(2.2.3) — If (derived_from<Cs, forward_iterator_tag> && ...) is true, iterator_category denotes forward_iterator_tag.

(2.2.4) — Otherwise, iterator_category denotes input_iterator_tag.

```
template <std::size_t N>
constexpr void satisfy();                              // exposition only
```

3 *Effects*: Equivalent to:

```
if constexpr (N != (sizeof...(Views) - 1)) {
    if (get<N>(it_) == ranges::end(get<N>(parent_->views_))) {
        it_.template emplace<N + 1>(ranges::begin(get<N + 1>(parent_->views_)));
        satisfy<N + 1>();
    }
}
```

```
template <std::size_t N>
constexpr void prev();                                 // exposition only
```

4 *Effects*: Equivalent to:

```
if constexpr (N == 0) {
    --get<0>(it_);
} else {
    if (get<N>(it_) == ranges::begin(get<N>(parent_->views_))) {
        using prev_view = maybe-const<Const, tuple_element_t<N - 1, tuple<Views...>>>;
        if constexpr (common_range<prev_view>) {
            it_.template emplace<N - 1>(ranges::end(get<N - 1>(parent_->views_)));
        } else {
            it_.template emplace<N - 1>(
                ranges::next(ranges::begin(get<N - 1>(parent_->views_)),
                             ranges::size(get<N - 1>(parent_->views_))));
        }
        prev<N - 1>();
    } else {
        --get<N>(it_);
    }
}
```

```
template <std::size_t N>
constexpr void advance_fwd(difference_type offset, difference_type steps); // exposition only
```

5 *Effects*: Equivalent to:

```
if constexpr (N == sizeof...(Views) - 1) {
    get<N>(it_) += steps;
} else {
    auto n_size = ranges::size(get<N>(parent_->views_));
    if (offset + steps < static_cast<difference_type>(n_size)) {
        get<N>(it_) += steps;
    } else {
        it_.template emplace<N + 1>(ranges::begin(get<N + 1>(parent_->views_)));
        advance_fwd<N + 1>(0, offset + steps - n_size);
    }
}
```

```
template <std::size_t N>
constexpr void advance_bwd(difference_type offset, difference_type steps); // exposition only
```

6 *Effects*: Equivalent to:

```
if constexpr (N == 0) {
    get<N>(it_) -= steps;
} else {
    if (offset >= steps) {
        get<N>(it_) -= steps;
    } else {
        it_.template emplace<N - 1>(ranges::begin(get<N - 1>(parent_->views_)) +
                                    ranges::size(get<N - 1>(parent_->views_)));
        advance_bwd<N - 1>(
            static_cast<difference_type>(ranges::size(get<N - 1>(parent_->views_))),
            steps - offset);
    }
}
```

```
template <class... Args>
explicit constexpr iterator(
            maybe-const<Const, concat_view>* parent,
            Args&&... args)
    requires constructible_from<base_iter, Args&&...>; // exposition only
```

7 *Effects*: Initializes *parent_* with `parent`, and initializes *it_* with `std::forward<Args>(args)...`.

```
constexpr iterator(iterator<!Const> i)
    requires Const &&
    (convertible_to<iterator_t<Views>, iterator_t<maybe-const<Const, Views>>>&&...);
```

8 *Effects*: Initializes *parent_* with `i.`*parent_*, and initializes *it_* with `std::move(i.`*it_*`)`.

```
constexpr decltype(auto) operator*() const;
```

9 *Preconditions*: *it_*`.valueless_by_exception()` is `false`.

10 *Effects*: Equivalent to:

```
using reference = common_reference_t<range_reference_t<maybe-const<Const, Views>>...>;
return std::visit([](auto&& it) -> reference {
    return *it; }, it_);
```

```
constexpr iterator& operator++();
```

11 *Preconditions*: *it_*`.valueless_by_exception()` is `false`.

12 *Effects*: Let *i* be *it_*`.index()`. Equivalent to:

```
++get<i>(it_);
satisfy<i>();
return *this;
```

```
constexpr void operator++(int);
```

13 *Preconditions*: `it_.valueless_by_exception()` is `false`.

14 *Effects*: Equivalent to:

```
++*this;
```

```
constexpr iterator operator++(int)
    requires(forward_range<maybe-const<Const, Views>>&&...);
```

15 *Preconditions*: `it_.valueless_by_exception()` is `false`.

16 *Effects*: Equivalent to:

```
auto tmp = *this;
++*this;
return tmp;
```

```
constexpr iterator& operator--()
    requires concat-bidirectional<maybe-const<Const, Views>...>;
```

17 *Preconditions*: `it_.valueless_by_exception()` is `false`.

18 *Effects*: Let *i* be *it_.index()*. Equivalent to:

```
prev<i>();
return *this;
```

```
constexpr iterator operator--(int)
    requires concat-bidirectional<maybe-const<Const, Views>...>
```

19 *Preconditions*: `it_.valueless_by_exception()` is `false`.

20 *Effects*: Equivalent to:

```
auto tmp = *this;
--*this;
return tmp;
```

```
constexpr iterator& operator+=(difference_type n)
    requires concat-random-access<maybe-const<Const, Views>...>;
```

21 *Preconditions*: `it_.valueless_by_exception()` is `false`.

22 *Effects*: Let *i* be *it_.index()*. Equivalent to:

```
if(n > 0) {
  advance_fwd<i>(get<i>(it_) - ranges::begin(get<i>(parent_->views_)), n);
} else if (n < 0) {
  advance_bwd<i>(get<i>(it_) - ranges::begin(get<i>(parent_->views_)), -n);
}
return *this;
```

```
constexpr iterator& operator-=(difference_type n)
    requires concat-random-access<maybe-const<Const, Views>...>;
```

23 *Preconditions*: `it_.valueless_by_exception()` is `false`.

24 *Effects*: Equivalent to:

```
*this += -n;
return *this;
```

```
constexpr decltype(auto) operator[](difference_type n) const
    requires concat-random-access<maybe-const<Const, Views>...>;
```

25 *Preconditions*: `it_.valueless_by_exception()` is `false`.

26 *Effects*: Equivalent to:

```
    return *((*this) + n);
```

```
friend constexpr bool operator==(const iterator& x, const iterator& y)
    requires(equality_comparable<iterator_t<maybe-const<Const, Views>>>&&...);
```

27 *Preconditions*: `x.it_.valueless_by_exception()` and `y.it_.valueless_by_exception()` are each `false`.

28 *Effects*: Equivalent to:

```
    return x.it_ == y.it_;
```

```
friend constexpr bool operator==(const iterator& it, default_sentinel_t);
```

29 *Preconditions*: `it.it_.valueless_by_exception()` is `false`.

30 *Effects*: Equivalent to:

```
    constexpr auto last_idx = sizeof...(Views) - 1;
    return it.it_.index() == last_idx &&
           get<last_idx>(it.it_) == ranges::end(get<last_idx>(it.parent_->views_));
```

```
friend constexpr bool operator<(const iterator& x, const iterator& y)
    requires(random_access_range<maybe-const<Const, Views>>&&...);
```

31 *Preconditions*: `x.it_.valueless_by_exception()` and `y.it_.valueless_by_exception()` are each `false`.

32 *Effects*: Equivalent to:

```
    return x.it_ < y.it_;
```

```
friend constexpr bool operator>(const iterator& x, const iterator& y)
    requires(random_access_range<maybe-const<Const, Views>>&&...);
```

33 *Preconditions*: `x.it_.valueless_by_exception()` and `y.it_.valueless_by_exception()` are each `false`.

34 *Effects*: Equivalent to:

```
    return y < x;
```

```
friend constexpr bool operator<=(const iterator& x, const iterator& y)
    requires(random_access_range<maybe-const<Const, Views>>&&...);
```

35 *Preconditions*: `x.it_.valueless_by_exception()` and `y.it_.valueless_by_exception()` are each `false`.

36 *Effects*: Equivalent to:

```
    return !(y < x);
```

```
friend constexpr bool operator>=(const iterator& x, const iterator& y)
    requires(random_access_range<maybe-const<Const, Views>>&&...);
```

37 *Preconditions*: `x.it_.valueless_by_exception()` and `y.it_.valueless_by_exception()` are each `false`.

38 *Effects*: Equivalent to:

```
    return !(x < y);
```

```
friend constexpr auto operator<=>(const iterator& x, const iterator& y)
    requires((random_access_range<maybe-const<Const, Views>> &&
     three_way_comparable<maybe-const<Const, Views>>)&&...);
```

39 *Preconditions*: `x.it_.valueless_by_exception()` and `y.it_.valueless_by_exception()` are each `false`.

40 *Effects*: Equivalent to:

```
return x.it_ <=> y.it_;
```

```
friend constexpr iterator operator+(const iterator& it, difference_type n)
    requires concat-random-access<maybe-const<Const, Views>...>;
```

41 *Preconditions*: `it.it_.valueless_by_exception()` is `false`.

42 *Effects*: Equivalent to:

```
return iterator{it} += n;
```

```
friend constexpr iterator operator+(difference_type n, const iterator& it)
    requires concat-random-access<maybe-const<Const, Views>...>;
```

43 *Preconditions*: `it.it_.valueless_by_exception()` is `false`.

44 *Effects*: Equivalent to:

```
return it + n;
```

```
friend constexpr iterator operator-(const iterator& it, difference_type n)
    requires concat-random-access<maybe-const<Const, Views>...>;
```

45 *Preconditions*: `it.it_.valueless_by_exception()` is `false`.

46 *Effects*: Equivalent to:

```
return iterator{it} -= n;
```

```
friend constexpr difference_type operator-(const iterator& x, const iterator& y)
    requires concat-random-access<maybe-const<Const, Views>...>;
```

47 *Preconditions*: `x.it_.valueless_by_exception()` and `y.it_.valueless_by_exception()` are each `false`.

48 *Effects*: Let i_x denote `x.it_.index()` and i_y denote `y.it_.index()`

(48.1) — if $i_x > i_y$, let d_y denote the distance from `get<`i_y`>(y.it_)` to the end of `get<`i_y`>(y.`*`parent_`*`.`*`views_`*`)`, d_x denote the distance from the begin of `get<`i_x`>(x.`*`parent_`*`.`*`views_`*`)` to `get<`i_x`>(x.`*`it_`*`)`. For every integer $i_y < i < i_x$, let `s` denote the sum of the sizes of all the ranges `get<`*`i`*`>(x.`*`parent_`*`.`*`views_`*`)` if there is any, and `0` otherwise, equivalent to

```
return d_y + s + d_x;
```

(48.2) — otherwise, if $i_x < i_y$, equivalent to:

```
return -(y - x);
```

(48.3) — otherwise, equivalent to:

```
return get<i_x>(x.it_) - get<i_y>(y.it_);
```

```
friend constexpr difference_type operator-(const iterator& x, default_sentinel_t)
    requires concat-random-access<maybe-const<Const, Views>...>;
```

49 *Preconditions*: `x.it_.valueless_by_exception()` is `false`.

50 *Effects*: Let i_x denote `x.it_.index()`, d_x denote the distance from `get<`i_x`>(x.it_)` to the end of `get<`i_x`>(x.`*`parent_`*`.`*`views_`*`)`. For every integer $i_x < i <$ `sizeof...(Views)`, let `s` denote the sum of the sizes of all the ranges `get<`*`i`*`>(x.`*`parent_`*`.`*`views_`*`)` if there is any, and `0` otherwise, equivalent to

```
return -(d_x + s);
```

```
friend constexpr difference_type operator-(default_sentinel_t, const iterator& x)
    requires concat-random-access<maybe-const<Const, Views>...>;
```

51 *Preconditions*: `x.it_.valueless_by_exception()` is `false`.

52 *Effects*: Equivalent to:

```
return -(x - default_sentinel);
```

```
friend constexpr decltype(auto) iter_move(iterator const& it) noexcept(see below);
```

53 *Preconditions*: `it.it_.valueless_by_exception()` is `false`.

54 *Effects*: Equivalent to:

```
return std::visit(
    [](auto const& i) ->
        common_reference_t<range_rvalue_reference_t<maybe-const<Const, Views>>...> {
        return ranges::iter_move(i);
    },
    it.it_);
```

55 *Remarks*: The exception specification is equivalent to:

```
((is_nothrow_invocable_v<decltype(ranges::iter_move),
                         const iterator_t<maybe-const<Const, Views>>&> &&
  is_nothrow_convertible_v<range_rvalue_reference_t<maybe-const<Const, Views>>,
                           common_reference_t<range_rvalue_reference_t<
                             maybe-const<Const, Views>>...>>) &&...)
```

```
friend constexpr void iter_swap(const iterator& x, const iterator& y) noexcept(see below)
    requires see below;
```

56 *Preconditions*: `x.it_.valueless_by_exception()` and `y.it_.valueless_by_exception()` are each `false`.

57 *Effects*: Equivalent to:

```
std::visit(ranges::iter_swap, x.it_, y.it_);
```

58 *Remarks*: The exception specification is `true` if and only if: For every combination of two types `X` and `Y` in the set of all types in the parameter pack `iterator_t<maybe-const<Const, Views>>>...`, `is_nothrow_invocable_v<decltype(ranges::iter_swap), const X&, const Y&>` is true.

59 *Remarks*: The expression in the requires-clause is `true` if and only if: For every combination of two types `X` and `Y` in the set of all types in the parameter pack `iterator_t<maybe-const<Const, Views>>>...`, `indirectly_swappable<X, Y>` is modelled.

§ 5.4 Feature Test Macro

Add the following macro definition to 17.3.2 [version.syn], header `<version>` synopsis, with the value selected by the editor to reflect the date of adoption of this paper:

```
#define __cpp_lib_ranges_concat  20XXXXL // also in <ranges>
```

§ 6 References

[ours] Hui Xie and S. Levent Yilmaz. A proof-of-concept implementation of views::concat. https://github.com/huixie90/cpp_papers/tree/main/impl/concat

[P2214R1] Barry Revzin, Conor Hoekstra, Tim Song. 2021-09-14. A Plan for C++23 Ranges. https://wg21.link/p2214r1

[P2278R1] Barry Revzin. 2021-09-15. cbegin should always return a constant iterator.
https://wg21.link/p2278r1

[P2321R2] Tim Song. 2021-06-11. zip.
https://wg21.link/p2321r2

[P2374R3] Sy Brand, Michał Dominiak. 2021-12-13. views::cartesian_product.
https://wg21.link/p2374r3

[range-v3] Eric Niebler. range-v3 library.
https://github.com/ericniebler/range-v3